

Practical - 8

Aim: Implementation of Graph and Searching (DFS and BFS).

Algorithm :

Input : V - no. of vertices
E - No. of edges
E pairs of vertices (u,v) representing edges

Output : DFS travel sequence
BFS travel sequence
Execution time of DFS (in nanoseconds)
Execution time of BFS (in nanoseconds)

Algorithm:

- ① Start
- ② Read the number of vertices V
- ③ Create an adjacency list with V empty lists.
- ④ Read the number of edges E
- ⑤ Repeat E times :
 - read vertices u and v
 - add v to the adjacent list of u
 - add u to the adjacent list of v
- ⑥ Read the starting vertex.
- ⑦ DFS Traversal : create a visited array and initialize all values to false
- ⑧ Mark the current vertex as visited and print it
- ⑨ visit each adjacent vertex recursively if it has not been visited.
- ⑩ Continue all reachable vertices are visited.
- ⑪ Stop the DFS Timer and store the execution time.
- ⑫ BFS Traversal : Reset the visited array to false
- ⑬ Create an empty queue.
- ⑭ Mark the starting vertex as visited and insert it into queue.
- ⑮ Repeat until the queue becomes empty
 - remove the front vertex from the queue and print removed vertex
 - visit each adjacent vertex
 - If a neighbour is not visited, mark it as visited & insert into queue.
- ⑯ Display DFS, BFS traversal and execution time in nanoseconds.
- ⑰ Stop.



Program:

```
#include<iostream>
using namespace std;

class Graph {

public:
    int adj[10][10]; // adjacency matrix
    int visited[10]; // visited array
    int n;           // number of vertices

    // — Get Input —
    void getInput() {
        cout << "Enter number of vertices : ";
        cin >> n;

        cout << "Enter adjacency matrix (" << n << "x" << n << ") : " << endl;
        for(int i = 0; i < n; i++)
            for(int j = 0; j < n; j++)
                cin >> adj[i][j];
    }

    // — Reset Visited —
    void resetVisited() {
        for(int i = 0; i < n; i++)
            visited[i] = 0;
    }

    void BFS(int start) {
        int queue[10];
        int front = 0, rear = 0;

        resetVisited();

        // enqueue start vertex
        queue[rear++] = start;
        visited[start] = 1;

        cout << "BFS Traversal : ";

        while(front < rear) {

            // dequeue
            int node = queue[front++];
            cout << node << " ";

            // visit all unvisited neighbours
            for(int i = 0; i < n; i++) {
                if(adj[node][i] == 1 && visited[i] == 0) {
                    queue[rear++] = i;
                    visited[i] = 1;
                }
            }
        }
    }
};
```



```
    }
    }
    }
    cout << endl;
}

void DFS(int node) {
    visited[node] = 1;
    cout << node << " ";

    for(int i = 0; i < n; i++) {
        if(adj[node][i] == 1 && visited[i] == 0) {
            DFS(i); // recursive call
        }
    }
}

// — Start DFS —
void startDFS(int start) {
    resetVisited();
    cout << "DFS Traversal : ";
    DFS(start);
    cout << endl;
}

// — Display Adjacency Matrix —
void displayMatrix() {
    cout << "\nAdjacency Matrix:" << endl;
    cout << " ";
    for(int i = 0; i < n; i++)
        cout << i << " ";
    cout << endl;

    for(int i = 0; i < n; i++) {
        cout << i << " ";
        for(int j = 0; j < n; j++)
            cout << adj[i][j] << " ";
        cout << endl;
    }
}

};

// — MAIN —
int main() {

    Graph g; // object

    g.getInput();
    g.displayMatrix();

    int start;

    Name: Vemalatha Bhimireddy
    Enrollment Number:92460118162
```



```
cout << "\nEnter starting vertex : ";  
cin >> start;
```

```
cout << endl;  
g.BFS(start);  
g.startDFS(start);
```

```
return 0;
```

```
}
```

Output :

```
PS C:\SEM5\DAA\PRACTICALS> g++ .\bfs_dfs.cpp -o bfs  
PS C:\SEM5\DAA\PRACTICALS> .\bfs.exe  
Enter number of vertices: 5  
Enter number of edges: 4  
Enter edges (u v):  
0 1  
0 2  
1 3  
1 4  
Enter starting vertex: 0  
  
DFS Traversal: 0 1 3 4 2  
  
BFS Traversal: 0 1 2 3 4  
  
Execution Time:  
DFS: 3002000 ns  
BFS: 999000 ns
```

Time complexity :

Time complexity

```

get Input ()
for (i=0; i<n; i++) {  $\rightarrow O(n)$ 
    for (j=0; j<n; j++) {  $\rightarrow O(n)$ 
         $O(n) \Rightarrow adj[i][j]$ ;
    }
}

```

$\therefore O(n) \times O(n) = O(n^2)$

BFS:- resetVisited()

```

for (i=0; i<n; i++)  $\rightarrow O(n)$ 
while (front < rear) {  $\rightarrow O(n)$ 
    for (i=0; i<n; i++)  $\rightarrow O(n)$ 
    ...
}
}

```

Total Time complexity = $O(n) + O(n) \times O(n)$
 $= O(n) + O(n^2) = O(n^2)$

DFS :
DFS (node)

```

for (i=0; i<n; i++) {  $\rightarrow O(n)$ 
    {
        if (---)
            DFS(i);  $\rightarrow$  recursive call
    }
}

```

DFS visits at most n vertices $\rightarrow O(n)$

Time complexity: $O(n) \times O(n) = O(n^2)$

Time complexity (BFS & DFS)

Best case - $O(n^2)$: The minimum required matrix checks still take quadratic time.

Average case - $O(n^2)$: BFS and DFS produces vertices and their possible connections, resulting in quadratic time.

Worst case: $O(n^2)$: All vertices are visited and all possible connections are checked, resulting in quadratic time.

Conclusion:

Thus, BFS and DFS graph traversal algorithms were successfully implemented and executed using an adjacency matrix. BFS traverses the graph level by level, while DFS traverses the graph depth-wise. The time complexity of both algorithms for the given adjacency matrix implementation is $O(n^2)$.